CPSC 120L, Intro to Programming
Lab 7: Input Validation and Boolean Logic

Student Learning Objectives:



In this lab, you will gain familiarity with and practice:

- Using Compound Boolean Expressions
- Validating Program and Function Input
- Gaining Nested and Chained `if` Statement Proficiency
- Writing well formatted programs
- Using and multi-translation unit coding solutions with C++ modules
- Collaborating in pairs using the pair programming software development technique, switching roles frequently

You are provided with a point-of-departure baseline centered around parking your car alongside the street curb. In this exercise you will write functions that implement parking restriction rules posted on signs. This code could be used in a system that automates parking rules, such as an app that tells you where you are allowed to park, or an app that parking enforcement officers use to issue parking tickets.

Lab Instructions:

This assignment shall be completed by pairs[1] of students using the pair programming software development technique. If forming a new team, choose a partner you have not partnered with before. Priority should be given to completing this lab during your scheduled lab session, but it shall be completed as a team before the assignment closes as specified in Canvas. Students shall continue collaborating within their teams outside of class. You may use either a machine in the CS Lab or your own machine but remember lab machines are reset and work not saved elsewhere (GitHub, flash drive, etc.) will be lost. Also remember that all team members must have write access to the team's work. This lab assignment assumes you have already successfully completed all prior labs.

1. Use a web browser to open this assignment in Canvas. You'll find a link to the corresponding Gradescope assignment and a link to the point-of-departure student files.
   a. Open the Gradescope assignment and form your Gradescope group making sure everyone in your team accepts their emailed invitation. If forming a new team, choose a partner you have not partnered with before. Do this first thing in class before leaving the room. See Adding Group Members to a Submission – Gradescope Guides.
      i. Viewing and editing group members is done through the submission page, but that page isn't accessible until someone from the group submits a file. Then that person adds the other members.
      ii. What file should you submit in this initial group-forming setup step? It doesn't really matter. Use a file from a prior assignment, a text file you have laying around, even a pdf file will work. The expected result will be your submission is inspected by the autograder and rejected. Rejected submissions do not count against you. But you gain access to the submission page where you can form groups.
   b. Make an entry in your notebook for this lab. Write your team member's contact information, as well as the exact physical (personal or lab) machine you are using. Log your steps (what and when) with very detailed descriptions (screenshots, pictures, etc.) of any errors you encounter.
   c. Download your point-of-departure student files into your Downloads (~/Downloads) directory
   d. Unzip your point-of-departure student files into your C++ Development Root directory and verify the contents. Refer to Lab 1 if you forgot how to do that.

[1] One team may have 3 students if there are an odd number of students in the class. In this case, it is better to have more team members than doing the lab solo.

e. Attempt to build your solution. This may not build successfully, but doing this before you open the text editor for the first time will configure the build so the text editor in the next steps can better understand your code.

2. Read and understand the concepts in Appendices A and B

3. main.cpp

   a. Function main(…) is the entry point to your program and orchestrates the operator's query. Prompt for when (day, hour, minute) and where (street) the operator wishes to park. Validate the operator's information. If the information is invalid, reject the request and display an explanation. Otherwise, process the request and display either allowed, not allowed, or no information for the requested street available.

4. Lab7.cppm

   a. Functions isValidDay(…), isValidHour(…), and isValidMinute(…) take one argument and return true if the day, hour or minute respectively are within normal ranges, and false otherwise. Days must be fully spelled and be all lower case characters except for the first character which must be capitalized. Hours follow military time and must be less than 24, while minutes must be less than 60.

   b. Functions canParkOnAsh(…), canParkOnBeech, canParkOnCedar, take two arguments (day and hour) and function canParkOnDate(…) takes three arguments (day, hour, minute). All return a value indicating parking is either allowed or not allowed. Each street has its own parking rules as described in Appendix A.

5. MyLibrary.cppm

   a. This module is carried over from prior labs. Verify it is working correctly and if not, make corrections. Be careful, just because it "worked" last time doesn't mean it will work this time - sometimes bugs don't show up until new test cases are discovered. If you don't have this from prior labs, you will have to complete it for this lab.

6. Make sure you are in the project's root directory (the directory where `main.cpp` resides) then build and run your project using redirection. Build with

```
$>  build
```

   and run with

```
$>  bin/project_clangxx_Release | tee output.txt
```

   Notice the output of your program, including user prompts, is captured in the `output.txt` file. However, the user's responses are not captured. This is to be expected since your program did not write those. Notice also the test results are still written to your screen but not captured in the output.txt file This is also expected. We want to capture only the output from your program in the `output.txt` file.

7. Submit your completed solution to Gradescope for credit. After a short while Gradescope will respond with feedback. If necessary, make corrections and, within limits, submit again for an improved score.

Deliverable Artifacts:

Provided files	Files to deliver	Comments
main.cpp Lab7.cppm	1. main.cpp 2. Lab7.cppm	Start with the files provided. Make changes in the designated TO-DO sections (only). The grading process will detect and discard all other changes.
MyLibrary.cppm	3. MyLibrary.cppm	Replace with your (potentially updated) file from a previous assignment.
sample_output.txt sample_run.txt	4. output.txt	Capture your program's output to this text file using command line redirection. See command redirection. Failure to deliver this file indicates you could not get your program to execute. Screenshots or terminal window log files are not permitted.
_clang-format	5. _clang-format	You need to deliver this file, but there is nothing you *have* to do. You can optionally modify this file with formatting style overrides. See Formatting.
	Readme.txt	Optional. Use it to communicate your thoughts to the grader.
RegressionTests/*		When you're far enough along and ready to have your functions tested, then place this folder in your working directory. These tests will be added to your delivery and executed during the grading process. The grading process expects all tests to pass.

Appendix A

Parking Rules

As you may know, some city streets allow cars to park alongside the curb at all times, while other streets have signs that restrict when parking is allowed. Parking may be restricted to make way for street sweeping, landscaping, farmers markets, and other civic routines. Sometimes there can be multiple signs that make for complicated rules.

In this exercise you will write functions that implement the parking restriction rules on signs. This code could be used in a system that automates parking rules, such as an app that tells you where you are allowed to park, or an app that parking enforcement officers use to issue parking tickets.

Our city has four streets, all named after trees: Ash Street, Beech Street, Cedar Street, and Date Street. Each street has a different parking sign. The parking rules depend on:

- The day of the week;
- the hour and minute of the day.

You should assume parking is allowed unless there is a sign giving specific times with parking is not allowed. Parking is not allowed starting at the time posted and remains not allowed up to *but not including* the ending time posted. If, for example, the sign says parking is not allowed from 10 AM to 12PM (noon), then parking is allowed at 9:59 AM but not at 10:00 AM. Parking is not allowed at 11:59 AM but is allowed at 12:00 PM.

For each street, you will write a function that decides whether parking is allowed, or not allowed, at a specific time.

Signage

Street parking rules are as follows:

			
Ash Street	Beech Street	Cedar Street	Date Street

Appendix B

Rules and Constraints:

1. Do not leave the laboratory until you have completed the lab. If you complete the lab early, demonstrate your lab to your instructor and ask for permission to leave early.
2. Students should switch roles every 15 minutes or whenever directed by the instructor. In this lab, it's recommended to take turns using the same computer.
3. Gradescope submissions apply to the entire group regardless of who actually submitted. All group members can submit, but only the last one will be graded for the entire group. Submissions are not cumulative, each must stand alone.
4. You are to modify only designated TO-DO sections using only the techniques discussed in class. The grading process will detect and discard any changes made outside the designated TO-DO sections, including spacing and formatting. Designated TO-DO sections are identified with the following comments:

   ```
   ////////////////////////// TO-DO (X) ////////////////////////////////
   ...
   ////////////////////////// END-TO-DO (X) ////////////////////////////
   ```

   Keep and do not alter these comments. Insert your code between them.
5. Avoid inserting complex expressions into streams. Instead, use the standard format and standard print functions. Instead of `std::cout << … << … <<` and so on, use `std::print( std::cout, "…", …, …,` and so on.
6. Avoid the *to_string(…)* like functions. Instead, use `std::format("…", …, …, …)`.
7. Avoid `#include` directives. We are using C++ Modules exclusively, which are imported not included. Rare exceptions are used in very specific circumstances and are specifically noted.

Reminders:

- Multiple submissions may be available and a waiting period between submissions may be enforced.
  - Submitting a Code assignment – Gradescope Guides
  - Troubleshooting Submission Upload Issues – Gradescope Guides
- A clean compile is an entrance criterion. Deliveries that do meet the entrance criteria cannot be graded.
- Use Build.sh (`build`) to compile and link your program. If the `build` command is not found, you may need to initialize your environment.
  - See Installing and Configuring Linux, especially the One Click Setup Command
- Filenames are case sensitive, both in source code and in your OS file system. Windows doesn't care about filename case, but Linux does.
- You may redirect standard input from a text file, but you must redirect standard output to a text file named output.txt. Failure to include output.txt in your delivery indicates you could not execute your program and will be scored accordingly. A screenshot of your terminal window is not acceptable. See How to use command redirection under Linux if you are unfamiliar with command line redirection.

Formatting

We write code to be human readable. The machine doesn't care if we indent, use blank lines, or put curly braces on the same line. But humans care greatly. Experts will attest code is read far more often than it is written. Well formatted code is a professional necessity that directly impacts readability, maintainability, collaboration, and code quality. Consistent formatting and style is one of the most important aspects of well-written code. So, what does well-written code look like?

Unknown. Put three programmers in a room and you'll have four opinions. There is a plethora of guidelines available, including the C++ Core Guidelines, the LLVM Coding Standards, and the Google C++ Style Guide. Use our C++ Programming Style Guide as your starting point. In general, the clang-format file provided to you in the C++ Development Root follows the LLVM guide tailored with C++ Core Guidelines and the Allman brace style. This clang-format file embodies the baseline used to grade your work. Whatever formatting style you choose, the imperative rule is that you be consistent.

Included in your point-of-departure student files is a file called `_clang_format`. The contents of this file overrides the baseline used to grade your work. It's intended to offer you some flexibility in choosing your style. If, for example, you can't live with the Allman brace style, you may pick another brace style by setting the `BreakBeforeBraces` option to something else, say `Attach`. For a complete set of options to tweak, see Clang-Format Style Options.

That said, there is nothing you *need* to do. Simply submit `_clang_format` unchanged with all your other files. But if you want to be graded against a different style, submit your modified `_clang_format` file. Keep in mind that no tool will ever be able to format your code perfectly. Formatting tools are a starting point. Ultimately, you the human must ensure your code is of high quality, readable, and maintainable.

Testing

Computers only ever do exactly what they are told, exactly the way they are told it, and never anything else. Testing is an important process in writing a program. You need to test for 1) the program to behave correctly and 2) that the program behaves incorrectly in a predictable way.

As a strategy, start simple and work your way up. Good tests are specific, cover a broad range of fundamentally different possibilities, can identify issues quickly, easily, and directly, without need for much set up, and can almost be diagnosed by inspection if the code fails to execute the test correctly. Ideally, one would write tests that verify behavior, results, and constraints before any solution code is written. This is known as test-driven development.

Included with your point-of-departure student files is a subdirectory named `RegressionTests`. There is nothing in there you need to do. Its purpose is to provide some basic tests to help you know when your project is "working". This is a just starting point. You are expected to test your program beyond these basic tests. We can test to find bugs, but we cannot test to find the absence of bugs. In other words, just because all your tests pass doesn't mean there are no bugs in your code. Automated testing is complemented with manual inspection which may reveal additions errors in your code.

The provided regression tests are built into your project and executed every time the project is executed. There is nothing special you need to do to invoke the tests. On the contrary, you may find in early stages of development these tests get in the way. So, disable them until you're ready, but do remember to reenable them before you're done.

To disable the regression tests, simply remove them from the build process by renaming `RegressionTests` to `.RegressionTests`. The build tools won't look inside hidden directories (directories that start with a dot). Use the move (mv) command to rename a file or directory. For example,

```
mv RegressionTests .RegressionTests
```

renames `RegressionTests` to `.RegressionTests`

To reenable them, rename it back.

```
mv .RegressionTests RegressionTests
```

If you want more control, rename an individual test to have a leading dot.

In addition, a shell script named `check_formatting.sh` is provided in `RegressionTests` subdirectory. You can use it to check your formatting at any time. For example, issuing

```
RegressionTests/check_formatting.sh main.cpp
```

will generate a formatting report. You can save the report to a file and then view the file with VSCode if that helps:

```
RegressionTests/check_formatting.sh main.cpp > formatting_report.txt
code formatting_report.txt
```

Tip: `less` tool less(1) - Linux manual page is helpful in a variety of situations to view the contents of a file, or more accurately the contents of a stream. A common idiom is to pipe the output of one command into the input of the next

command. Try viewing the output of `check_formatting.sh` with `less` and avoid creating `formatting_report.txt`, like this (type `q` to quit `less`)

```
RegressionTests/check_formatting.sh main.cpp | less
```

Acknowledgements

Special thanks go to the Computer Science staff at CSUF, especially Professors Michael Shafae, Keven Wortman, Doina Bain, ASM Rizvi, Spencer Moran, Garrett Whelan, and all the other prior CPSC 120 instructors who this work is based on.